
Спецификация

Компонент Time.C89

Статус: Черновик

Дата: Ноябрь 8, 2021

Версия: 1.0

Авторы	Компания
Владимир Башев	ПИРФ

Содержание

1. Обзор.....	3
1.1. Введение.....	3
1.2. Примечание.....	3
1.3. Ссылки.....	3
2. Компонент Eco.Time.C89.....	4
2.1. IEcoTimeC89 описание на ECO IDL.....	5
2.1.1. Функция clock.....	6
2.1.2. Функция difftime.....	6
2.1.3. Функция mktime.....	6
2.1.4. Функция time.....	6
2.1.5. Функция asctime.....	6
2.1.6. Функция ctime.....	7
2.1.7. Функция gmtime.....	7
2.1.8. Функция localtime.....	7
2.1.9. Функция strptime.....	7
Приложение А: Обучающие программы.....	8

1. Обзор

Данный документ описывает требования к реализации компонента Eco.Time.C89 (Компонент время).

1.1. Введение

Описание.

1.2. Примечание

- Ключевые слова в документе

1.3. Ссылки

Данный параграф содержит ссылки на информацию, помогающую понять данный документ:

[] – наименование ссылки

Доступен по: <http://адрес>

2. Компонент Eco.Time.C89

Компонент, имеет следующие описание:

2.1. IEcoTimeC89 описание на ECO IDL

ECO IDL

```
import "IEcoBase1.h"

[
object,
uguid(00000000-0000-0000-0000-890000007101),
]

interface IEcoTimeC89 : IEcoUnknown {

clock_t                cLock                ();

double                 difftime             ([in] time_t time1,
[in] time_t time2);

time_t                 mktime               ([in] struct tm * timeptr);

time_t                 time                 ([in] time_t * timer);

char *                  asctime             ([in] const struct tm * timeptr);

char *                  ctime               ([in] const time_t * timer);

struct tm *             gmtime              ([in] const time_t * timer);

struct tm *             localtime           ([in] const time_t * timer);

size_t                  strftime            ([in] char * str,
[in] size_t maxsize,
[in] const char * format,
[in] const struct tm * timeptr);

}
```

2.1.1. Функция clock

Библиотечная функция C возвращает количество тактов часов, прошедших с момента запуска программы. Чтобы получить количество секунд, используемых процессором, потребуется разделить на CLOCKS_PER_SEC.

В 32-битной системе, где CLOCKS_PER_SEC равно 1000000, эта функция будет возвращать одно и то же значение примерно каждые 72 минуты.

2.1.2. Функция difftime

Библиотечная функция C возвращает разницу секунд между time1 и time2, т.е. (time1 - time2). Эти два времени указываются в календарном времени, которое представляет время, прошедшее с момента Эпохи (00:00:00 1 января 1970 года по всемирному координированному времени (UTC)).

2.1.3. Функция mktime

Библиотечная функция C преобразует структуру, на которую указывает timeptr, в time_t значение в соответствии с местным часовым поясом.

2.1.4. Функция time

Библиотечная функция C возвращает время с начала эпохи (00:00:00 UTC, 1 января 1970 г.), измеряемое в секундах. Если seconds не равно NULL, возвращаемое значение также сохраняется в переменной seconds.

2.1.5. Функция asctime

Библиотечная функция C возвращает указатель на строку, представляющую день и время структуры struct timeptr.

2.1.6. Функция `ctime`

Библиотечная функция `C` возвращает строку, представляющую локальное время на основе аргумента `timer`.

Возвращаемая строка имеет следующий формат: `Www Mmm dd hh:mm:ss уууу`, где `Www` — день недели, `Mmm` — месяц буквами, `dd` — день месяца, `hh:mm:ss` — время и `уууу` — год.

2.1.7. Функция `gmtime`

Библиотечная функция `C` использует значение, указанное `timer`, для заполнения структуры `tm` значениями, представляющими соответствующее время, выраженное в часовом поясе UTC или GMT.

2.1.8. Функция `localtime`

Библиотечная функция `C` использует время, указанное `timer`, для заполнения структуры `tm` значениями, представляющими соответствующее локальное время. Значение таймера разбивается в структуру `tm` и выражается в местном часовом поясе.

2.1.9. Функция `strftime`

Библиотечная функция `C` форматирует время, представленное в структуре `timeptr` в соответствии с правилами форматирования, определенными в `format` и сохраненными в `str`.

Приложение А: Обучающие программы

```

/*
 * <кодировка символов>
 *   Cyrillic (UTF-8 with signature) - Codepage 65001
 * </кодировка символов>
 *
 * <сводка>
 *   EcoTimeC89
 * </сводка>
 *
 * <описание>
 *   Данный исходный файл является точкой входа обучающей программы по компоненту Eco.TimeC89
 * </описание>
 *
 * <автор>
 *   Copyright (c) 2018 Vladimir Bashev. All rights reserved.
 * </автор>
 *
 */

/* Eco OS */
#include "IEcoSystem1.h"
#include "IdEcoMemoryManager1.h"
#include "IdEcoInterfaceBus1.h"
#include "IdEcoTimeC89.h"
#include "IdEcoStdIOC89.h"
#include "IdEcoThreadManager1.h"

#define BST (+1)
#define CCT (+8)

/* Прототипы */
bool_t test_asctime(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO);
bool_t test_clock(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO);
bool_t test_ctime(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO);
bool_t test_difftime(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO, IEcoThread1* pICurrentThread);
bool_t test_gmtime(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO);
bool_t test_localtime(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO);
bool_t test_mktime(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO);
bool_t test_strftime(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO);
bool_t test_time(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO);

/*
 *
 * <сводка>
 *   Функция EcoMain
 * </сводка>
 *
 * <описание>
 *   Функция EcoMain - точка входа
 * </описание>
 *
 */

```

```

int16_t EcoMain(IEcoUnknown* pIUnk) {
    int16_t result = -1;
    /* Указатель на системный интерфейс */
    IEcoSystem1* pISys = 0;
    /* Указатель на интерфейс работы с системной интерфейсной шиной */
    IEcoInterfaceBus1* pIBus = 0;
    /* Указатель на интерфейс работы с памятью */
    IEcoMemoryAllocator1* pIMem = 0;
    char_t* name = 0;
    char_t* copyName = 0;
    /* Указатель на тестируемый интерфейс */
    IEcoTimeC89* pITime = 0;
    /* Указатель на интерфейс стандартного ввода/вывода */
    IEcoStdIOC89* pIStdIO = 0;
    /* Указатель на интерфейс работы с потоками */
    IEcoThreadManager1* pIThreadMgr = 0;
    IEcoThread1* pICurrentThread = 0;

    /* Проверка и создание системного интрефейса */
    if (pISys == 0) {
        result = pIUnk->pVTbl->QueryInterface(pIUnk, &IID_IEcoSystem1, (void **)&pISys);
        if (result != 0 && pISys == 0) {
            /* Освобождение системного интерфейса в случае ошибки */
            goto Release;
        }
    }

    /* Получение интерфейса для работы с интерфейсной шиной */
    result = pISys->pVTbl->QueryInterface(pISys, &IID_IEcoInterfaceBus1, (void **)&pIBus);
    if (result != 0 || pIBus == 0) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
}

#ifdef ECO_LIB
    /* Регистрация статического компонента для работы со временем */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &IID_EcoTimeC89,
(IEcoUnknown*)GetIEcoComponentFactoryPtr_0000000000000000000000000544D4531);
    if (result != 0 ) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
    /* Регистрация статического компонента для работы со стандартным вводом/выводом */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &IID_EcoStdIOC89,
(IEcoUnknown*)GetIEcoComponentFactoryPtr_000000000000000000000000053494F31);
    if (result != 0 ) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
    /* Регистрация статического компонента для работы с менеджером потоков */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &IID_EcoThreadManager1,
(IEcoUnknown*)GetIEcoComponentFactoryPtr_B7D8E10E1D3540CCB1CA96F766AF1936);
    if (result != 0 ) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
}
#endif

```

```

/* Получение интерфейса управления памятью */
result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoMemoryManager1, 0, &IID_IEcoMemoryAllocator1, (void**)
&pIMem);

/* Проверка */
if (result != 0 && pIMem == 0) {
    /* Освобождение системного интерфейса в случае ошибки */
    goto Release;
}

/* Выделение блока памяти */
name = (char_t *)pIMem->pVTbl->Alloc(pIMem, 10);

/* Заполнение блока памяти */
pIMem->pVTbl->Fill(pIMem, name, 'a', 9);

/* Получение тестируемого интерфейса */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoStdIOC89, 0, &IID_IEcoStdIOC89, (void**) &pIStdIO);
if (result != 0 && pIStdIO == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

/* Получение интерфейса для работы с менеджером потоков */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoThreadManager1, 0, &IID_IEcoThreadManager1, (void**)
&pIThreadMgr);
if (result != 0 && pIThreadMgr == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

/* Получение интерфейса текущего потока */
pICurrentThread = pIThreadMgr->pVTbl->get_CurrentThread(pIThreadMgr);

/* Получение тестируемого интерфейса */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoTimeC89, 0, &IID_IEcoTimeC89, (void**) &pITime);
if (result != 0 && pITime == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

/* Тестирование методов интерфейса */
test_asctime(pITime, pIStdIO);
test_clock(pITime, pIStdIO);
test_ctime(pITime, pIStdIO);
test_difftime(pITime, pIStdIO, pICurrentThread);
test_gmtime(pITime, pIStdIO);
test_localtime(pITime, pIStdIO);
test_mktime(pITime, pIStdIO);
test_strftime(pITime, pIStdIO);
test_time(pITime, pIStdIO);

/* Освобождение блока памяти */
pIMem->pVTbl->Free(pIMem, name);

Release:

/* Освобождение интерфейса для работы с интерфейсной шиной */

```

```

if (pIBus != 0) {
    pIBus->pVTbl->Release(pIBus);
}

/* Освобождение интерфейса работы с памятью */
if (pIMem != 0) {
    pIMem->pVTbl->Release(pIMem);
}

/* Освобождение тестируемого интерфейса */
if (pITime != 0) {
    pITime->pVTbl->Release(pITime);
}

/* Освобождение интерфейса работы со стандартным вводом/выводом */
if (pIStdIO != 0) {
    pIStdIO->pVTbl->Release(pIStdIO);
}

/* Освобождение интерфейса для работы с менеджером потоков */
if (pIThreadMgr != 0) {
    pIThreadMgr->pVTbl->Release(pIThreadMgr);
}

/* Освобождение системного интерфейса */
if (pISys != 0) {
    pISys->pVTbl->Release(pISys);
}

return result;
}

bool_t test_asctime(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO) {
    struct tm t;

    t.tm_sec    = 10;
    t.tm_min    = 10;
    t.tm_hour   = 6;
    t.tm_mday   = 25;
    t.tm_mon    = 2;
    t.tm_year   = 89;
    t.tm_wday   = 6;

    pIStdIO->pVTbl->puts(pIStdIO, pITime->pVTbl->asctime(pITime, &t));

    return 1;
}

bool_t test_clock(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO) {
    clock_t start_t, end_t;
    double total_t;
    int i;

    start_t = pITime->pVTbl->clock(pITime);
    pIStdIO->pVTbl->printf(pIStdIO, "Starting of the program, start_t = %ld\n", start_t);

    pIStdIO->pVTbl->printf(pIStdIO, "Going to scan a big loop, start_t = %ld\n", start_t);

```

```

for(i=0; i< 10000000; i++) {
}
end_t = pITime->pVTbl->clock(pITime);
pIStdIO->pVTbl->printf(pIStdIO, "End of the big loop, end_t = %ld\n", end_t);

total_t = (double)(end_t - start_t) / CLOCKS_PER_SEC;
pIStdIO->pVTbl->printf(pIStdIO, "Total time taken by CPU: %f\n", total_t );

return 1;
}

bool_t test_ctime(IEcoTimeC89* pITime, IEcoStdIOc89* pIStdIO) {
    time_t curtime;

    pITime->pVTbl->time(pITime, &curtime);

    pIStdIO->pVTbl->printf(pIStdIO, "Current time = %s", pITime->pVTbl->ctime(pITime, &curtime));

    return 1;
}

bool_t test_difftime(IEcoTimeC89* pITime, IEcoStdIOc89* pIStdIO, IEcoThread1* pICurrentThread) {
    time_t start_t, end_t;
    double diff_t;

    pIStdIO->pVTbl->printf(pIStdIO, "Starting of the program...\n");
    pITime->pVTbl->time(pITime, &start_t);

    pIStdIO->pVTbl->printf(pIStdIO, "Sleeping for 5 seconds...\n");
    pICurrentThread->pVTbl->Sleep(pICurrentThread, 5000);

    pITime->pVTbl->time(pITime, &end_t);
    diff_t = pITime->pVTbl->difftime(pITime, end_t, start_t);

    pIStdIO->pVTbl->printf(pIStdIO, "Execution time = %f\n", diff_t);
    pIStdIO->pVTbl->printf(pIStdIO, "Exiting of the program...\n");

    return 1;
}

bool_t test_gmtime(IEcoTimeC89* pITime, IEcoStdIOc89* pIStdIO) {
    time_t rawtime;
    struct tm *info;

    pITime->pVTbl->time(pITime, &rawtime);
    /* Get GMT time */
    info = pITime->pVTbl->gmtime(pITime, &rawtime);

    pIStdIO->pVTbl->printf(pIStdIO, "Current world clock:\n");
    pIStdIO->pVTbl->printf(pIStdIO, "London : %2d:%02d\n", (info->tm_hour+BST)%24, info->tm_min);
    pIStdIO->pVTbl->printf(pIStdIO, "China : %2d:%02d\n", (info->tm_hour+CCT)%24, info->tm_min);

    return 1;
}

bool_t test_localtime(IEcoTimeC89* pITime, IEcoStdIOc89* pIStdIO) {
    time_t rawtime;
    struct tm *info;

```

```

pITime->pVTbl->time(pITime, &rawtime);
info = pITime->pVTbl->localtime(pITime, &rawtime);
pIStdIO->pVTbl->printf(pIStdIO, "Current local time and date: %s", pITime->pVTbl->asctime(pITime, info));

return 1;
}

bool_t test_mktime(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO) {
    int ret;
    struct tm info;
    char buffer[80];

    info.tm_year = 2001 - 1900;
    info.tm_mon = 7 - 1;
    info.tm_mday = 4;
    info.tm_hour = 0;
    info.tm_min = 0;
    info.tm_sec = 1;
    info.tm_isdst = -1;

    ret = pITime->pVTbl->mktime(pITime, &info);
    if( ret == -1 ) {
        pIStdIO->pVTbl->printf(pIStdIO, "Error: unable to make time using mktime\n");
    } else {
        pITime->pVTbl->strftime(pITime, buffer, sizeof(buffer), "%c", &info );
        pIStdIO->pVTbl->printf(pIStdIO, buffer);
    }

    return 1;
}

bool_t test_strftime(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO) {
    time_t rawtime;
    struct tm *info;
    char buffer[80];

    pITime->pVTbl->time(pITime, &rawtime);

    info = pITime->pVTbl->localtime(pITime, &rawtime);

    pITime->pVTbl->strftime(pITime, buffer, 80, "%x - %I:%M%p", info);
    pIStdIO->pVTbl->printf(pIStdIO, "Formatted date & time : |%s|\n", buffer);

    return 1;
}

bool_t test_time(IEcoTimeC89* pITime, IEcoStdIOC89* pIStdIO) {
    time_t seconds;

    seconds = pITime->pVTbl->time(pITime, NULL);
    pIStdIO->pVTbl->printf(pIStdIO, "Hours since January 1, 1970 = %ld\n", seconds/3600);

    return 1;
}

```